Dokumentacja Techniczna Biblioteki Sieci Neuronowych

Poniższy dokument stanowi szczegółowe podsumowanie struktury oraz interfejsu programistycznego autorskiej biblioteki uczenia maszynowego stworzonej w czystym języku Python na potrzeby pracy inżynierskiej. Zawiera specyfikację funkcji wchodzących w skład modułu obliczeniowego, podział na moduły wstecznej propagacji, operacje wsadowe (batch) oraz przekształcenia danych.

1. Walidacja i Przetwarzanie Wstępne Danych

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`walidacja_danych(dane, etykiety)`	**Wejście:** `dane` (DataFrame), `etykiety` (lista kolumn) **Wyjście:** Brak (rzuca wyjątki `ValueError` / `KeyError`)	Sprawdza poprawność wejściowego zbioru danych. Weryfikuje, czy tablica nie jest pusta, czy wymagane kolumny docelowe istnieją w strukturze oraz czy w tabeli nie występują brakujące wartości (`NaN`).
`minmax(df, min_val, max_val)`	**Wejście:** `df` (DataFrame/Series), opcjonalne min/max **Wyjście:** Znormalizowany obiekt w zakresie [0, 1]	Przeprowadza skalowanie Min-Max cech wejściowych, sprowadzając wartości do przedziału jednostkowego w celu stabilizacji numerycznej procesu uczenia.
`odwroc_minmax(wartosci, min, max)`	**Wejście:** Znormalizowana wartość, oryginalne min i max **Wyjście:** Skalowana wartość w oryginalnej dziedzinie	Odwraca operację normalizacji Min-Max dla prognoz modelu, umożliwiając interpretację wyników w rzeczywistych jednostkach (np. USD).
`train_test_split(dane, etykiety, test, ziarno)`	**Wejście:** Dane, etykiety, udział testowy (np. 0.2), ziarno losowości **Wyjście:** `(dane_tr, dane_te, et_tr, et_te)`	Losowo dzieli zbiór danych na część treningową oraz testową z zachowaniem koherencji cech i etykiet, wspierając powtarzalność eksperymentów poprzez ziarno generatora.

2. Inicjalizacja Parametrów oraz Funkcje Aktywacji

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`stworz_wagi(dane, liczba_neuronow)`	**Wejście:** `dane_wejscia` (lista/macierz), `liczba_neuronow` (int) **Wyjście:** Dwuwymiarowa lista wag `[liczba_neuronow x liczba_wejsc]`	Inicjalizuje macierz wag początkowych w zadanym zakresie stochastycznym, dopasowując wymiar wejściowy na podstawie liczby cech próbki.
`stworz_biasy(liczba_neuronow)`	**Wejście:** `liczba_neuronow` (int) **Wyjście:** Jednowymiarowa lista biasów	Tworzy wektor początkowych wartości przesunięć (biasów) dla neuronów w warstwie.
`relu(x)` / `leaky_relu(x, alpha)`	**Wejście:** Wartość skalarna `x`, opcjonalny współczynnik `alpha` **Wyjście:** Przetworzona wartość aktywacji	Nieliniowe funkcje aktywacji. `relu` zeruje wartości ujemne, natomiast `leaky_relu` przepuszcza ułamek wartości ujemnych, zapobiegając problemowi martwych neuronów.

3. Propagacja w Przód (Forward Pass) oraz Funkcje Straty

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`neuron(wejscie, wagi, bias, akt)`	**Wejście:** Wektor wejść, wagi, bias, funkcja aktywacji **Wyjście:** Krotka `(z, a)` (suma liniowa oraz wynik aktywacji)	Oblicza sumę ważoną wejść powiększoną o bias (z) oraz aplikuje funkcję aktywacji (a) dla pojedynczego sztucznego neuronu.
`warstwa(wejscie, wagi, bias, akt)`	**Wejście:** Wektor wejść, macierz wag warstwy, biasy, aktywacja **Wyjście:** Krotka list `(z_lista, a_lista)`	Przetwarza całą warstwę neuronów dla pojedynczej próbki danych, iterując po jej składowych i agregując wyniki sum oraz aktywacji.
`mse(prognozy, prawdziwe)`	**Wejście:** Listy prognoz oraz wartości docelowych **Wyjście:** Skalarna wartość błędu średniokwadratowego (MSE)	Oblicza znormalizowany błąd średniokwadratowy pomiędzy wyjściem sieci a wzorcem dla całego zbioru lub batcha.
`pochodna_mse(y_pred, y_praw, N)`	**Wejście:** Przewidywana, prawdziwa wartość oraz rozmiar batcha `N` **Wyjście:** Skalarna wartość pochodnej błędu	Wyznacza gradient błędu względem wartości przewidywanej na podstawie kryterium straty MSE.

4. Wsteczna Propagacja (Backpropagation) i Aktualizacja Wag

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`cofaj_blad(...)`	**Wejście:** Błąd nastepny, wagi nastepne, z obecne, pochodna aktywacji **Wyjście:** Lista delt dla bieżącej warstwy	Propaguje sygnał błędu wstecz przez sieć dla pojedynczej próbki, wyznaczając błąd lokalny z uwzględnieniem wag połączeń i pochodnej aktywacji.
`aktualizuj_wagi(...)` / `aktualizuj_biasy(...)`	**Wejście:** Wejście, wagi, błąd, współczynnik uczenia (LR) **Wyjście:** Zaktualizowana macierz wag / wektor biasów	Modyfikuje parametry modelu w kierunku przeciwnym do gradientu z uwzględnieniem współczynnika uczenia (Gradient Descent).

5. Architektura Przetwarzania Wsadowego (Mini-Batch)

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`warstwa_batch(...)`	**Wejście:** Batch wejść, wagi, biasy, funkcja aktywacji **Wyjście:** Dwie listy list wsadowych `(batch_z, batch_a)`	Rozszerzenie propagacji w przód obsługujące paczkę próbek (batch) równolegle, gromadząc wartości sum i aktywacji w strukturach listowych.
`delta_batch_wyjscia(...)`	**Wejście:** Z, aktywacje, wartości y, rozmiar batcha, pochodne **Wyjście:** Dwuwymiarowa lista delt wyjściowych	Oblicza błąd początkowy dla całej paczki danych na warstwie wyjściowej przy użyciu funkcji straty i aktywacji.
`cofaj_blad_batch(...)`	**Wejście:** Batch delt wyższych, wagi, batch z, pochodna aktywacji **Wyjście:** Batch delt dla warstwy ukrytej	Wykonuje wsteczną propagację błędu dla wszystkich próbek w bieżącym batchu iteracyjnie.

Nazwa Funkcji & Sygnatura	Argumenty Wejściowe i Zwracane Wartości	Opis Działania
`aktualizuj_wagi_i_biasy_batch(...)`	**Wejście:** Batch wejść, batch delt, wagi, biasy, LR **Wyjście:** Zaktualizowane wagi i biasy	Akumuluje gradienty z całego batcha, dokonuje ich uśrednienia względem rozmiaru paczki i aktualizuje parametry sieci w jednym kroku optymalizacyjnym.